WEEK 1 – MACHINE LEARNING FUNDAMENTALS

Solved Assignments + Titanic Survival Prediction Mini Project

This solution is based on the uploaded Week 1 course sheet. The sheet covers ML fundamentals, dataset exploration, missing-data handling, encoding, scaling, train/test split, and a Titanic data-cleaning mini project. The assignment list appears on page 3 of the source file.

Source coverage: Week 1 topics and assignments are listed on pages 2–3 of the uploaded course PDF. The Titanic tasks are: clean missing data, encode Sex and Embarked, visualize age distribution, and output a cleaned CSV.

Assignment 1: Load a Dataset and Summarize Basic Statistics

Objective: Load a CSV dataset using Pandas and inspect its structure and statistical summary using `.info()` and `.describe()`.

Solution

```
import pandas as pd

# Load the dataset
df = pd.read_csv("titanic.csv")

# Display first five rows
print("First 5 rows:")
print(df.head())

# Display dataset structure, data types and non-null counts
print("\nDataset information:")
df.info()

# Display descriptive statistics for numerical columns
print("\nDescriptive statistics:")
print(df.describe())

# Optional: check the number of rows and columns
print("\nShape:", df.shape)

# Optional: check column names
print("\nColumns:")
print(df.columns.tolist())
```

Explanation: `read_csv()` loads the CSV into a DataFrame. `head()` previews records. `info()` shows the number of rows, columns, data types, and non-null values. `describe()` provides statistics such as count, mean, standard deviation, minimum, quartiles, and maximum for numeric columns.

Assignment 2: Handle Missing Data Using Mean/Median Imputation

Objective: Identify missing values and replace numerical missing values using mean or median imputation. For Titanic data, median imputation is generally useful for Age because it is less affected by extreme values.

Solution

```
# Check missing values
print("Missing values before imputation:")
print(df.isnull().sum())

# Mean imputation example
df["Age_mean"] = df["Age"].fillna(df["Age"].mean())

# Median imputation (recommended for the original Age column)
df["Age"] = df["Age"].fillna(df["Age"].median())

# Fill Embarked with its most frequent value (mode)
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Cabin has many missing values. A practical cleaning choice is
# to create a missingness indicator and then drop the original column.
df["Cabin_missing"] = df["Cabin"].isna().astype(int)
df = df.drop(columns=["Cabin"])

print("\nMissing values after imputation:")
print(df.isnull().sum())
```

Mean: sum of available values divided by the number of available values. **Median:** middle value after sorting. Median is often preferred when a variable is skewed or contains outliers. **Mode:** most frequent category and is commonly used for categorical data.

Important: Imputation values should ideally be calculated from the training set after splitting the data, when the goal is to build a predictive ML model. This prevents information from the test set from leaking into the training process.

Assignment 3: Encode Categorical Variables

Objective: Convert categorical variables into numerical representations using LabelEncoder and OneHotEncoder.

A. LabelEncoder

```
from sklearn.preprocessing import LabelEncoder
# Create a copy so the original DataFrame remains unchanged
```

```
df_label = df.copy()

le = LabelEncoder()
df_label["Sex"] = le.fit_transform(df_label["Sex"])

print(df_label[["Sex"]].head())
print("Classes:", le.classes_)
```

Note: LabelEncoder converts categories to integer labels, such as female → 0 and male → 1 (the exact mapping depends on the learned class order). It is suitable for a binary feature like Sex when the resulting numeric representation is acceptable, but it should not be used blindly for unordered multi-class features because the numbers can suggest an artificial ordering.

B. OneHotEncoder

```
from sklearn.preprocessing import OneHotEncoder

# Work on a fresh copy
df_ohe = df.copy()

encoder = OneHotEncoder(
    handle_unknown="ignore",
    sparse_output=False
)

encoded = encoder.fit_transform(df_ohe[["Sex", "Embarked"]])

encoded_df = pd.DataFrame(
    encoded,
    columns=encoder.get_feature_names_out(["Sex", "Embarked"]),
    index=df_ohe.index
)

# Remove original categorical columns and add encoded columns
df_ohe = df_ohe.drop(columns=["Sex", "Embarked"])
df_ohe = pd.concat([df_ohe, encoded_df], axis=1)

print(df_ohe.head())
```

Why One-Hot Encoding? It creates a separate binary column for each category. For example, Sex may become Sex_female and Sex_male. Embarked may become Embarked_C, Embarked_Q, and Embarked_S. This avoids imposing an artificial numerical order on nominal categories.

Mini Project 1: Titanic Survival Prediction – Data Cleaning

The uploaded Week 1 sheet specifies the Titanic Dataset (Kaggle) and four tasks: clean missing data, encode Sex and Embarked, visualize age distribution using Matplotlib/Seaborn, and output the cleaned dataset as a new CSV.

Complete End-to-End Solution

```
# ========================================
# TITANIC DATA CLEANING MINI PROJECT
# ========================================

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# 1. Load dataset
df = pd.read_csv("titanic.csv")

print("Initial shape:", df.shape)
print("\nInitial information:")
df.info()

print("\nInitial statistics:")
print(df.describe())

print("\nMissing values:")
print(df.isnull().sum())

# 2. Handle missing values

# Age: median imputation
df["Age"] = df["Age"].fillna(df["Age"].median())

# Embarked: mode imputation
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Cabin: preserve useful information as a missingness flag,
# then remove the high-missingness raw column.
df["Cabin_missing"] = df["Cabin"].isna().astype(int)
df = df.drop(columns=["Cabin"])

# 3. Encode Sex using LabelEncoder
label_encoder = LabelEncoder()
df["Sex"] = label_encoder.fit_transform(df["Sex"])

# 4. Encode Embarked using OneHotEncoder
one_hot = OneHotEncoder(
    handle_unknown="ignore",
    sparse_output=False
)

embarked_encoded = one_hot.fit_transform(df[["Embarked"]])

embarked_df = pd.DataFrame(
    embarked_encoded,
    columns=one_hot.get_feature_names_out(["Embarked"]),
    index=df.index
)

df = df.drop(columns=["Embarked"])
df = pd.concat([df, embarked_df], axis=1)

# 5. Visualize Age distribution
plt.figure(figsize=(8, 5))
sns.histplot(df["Age"], bins=30, kde=True)
plt.title("Age Distribution – Titanic Dataset")
plt.xlabel("Age")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()

# 6. Final missing-value check
print("\nMissing values after cleaning:")
print(df.isnull().sum())

# 7. Save cleaned dataset
output_file = "titanic_cleaned.csv"
df.to_csv(output_file, index=False)

print("\nCleaned dataset saved as:", output_file)
print("Final shape:", df.shape)
print("\nFinal columns:")
print(df.columns.tolist())
```

Expected Workflow

Step	Operation	Result
1	Load titanic.csv	DataFrame created
2	Inspect with info()/describe()	Structure and statistics understood
3	Check missing values	Null-value counts identified
4	Impute Age	Missing Age values replaced with median
5	Impute Embarked	Missing category replaced with mode
6	Handle Cabin	Missingness flag created; raw Cabin removed
7	Encode Sex	Binary numeric representation
8	Encode Embarked	One-hot binary columns
9	Plot Age	Histogram + KDE visualization
10	Export	titanic_cleaned.csv created

Installation Requirements

```
pip install pandas scikit-learn matplotlib seaborn
```

Important Version Note

Recent versions of scikit-learn use `sparse_output=False` in OneHotEncoder. If your installed version is older and reports an error for this argument, replace it with `sparse=False`.

How to Run

1. Download the Titanic CSV dataset and place it in the same folder as the Python script.
2. Rename the file to `titanic.csv` if necessary.
3. Install the required packages.
4. Run the script in VS Code, Jupyter Notebook, or another Python environment.
5. Check the generated age-distribution graph.
6. Confirm that `titanic_cleaned.csv` is created in the working folder.

Key Viva / Interview Questions

What is machine learning? A field in which algorithms learn patterns from data to make predictions or decisions without being explicitly programmed for every case.

What is supervised learning? Learning from labeled examples, where inputs are paired with known target outputs.

What is unsupervised learning? Finding patterns or structure in data without target labels, such as clustering.

What is reinforcement learning? Learning through interaction with an environment using rewards and penalties.

Why do we preprocess data? To improve data quality and make it suitable for machine-learning algorithms.

Why use median for Age? Median is more robust to skewed distributions and outliers than the mean.

Why encode categorical data? Most ML algorithms require numerical input, so categorical values must be represented numerically.

LabelEncoder vs OneHotEncoder? LabelEncoder maps categories to integer labels; OneHotEncoder creates separate binary columns for categories.

Why split train and test data? To evaluate how well a model generalizes to unseen data.

What is data leakage? When information unavailable at prediction time, often from validation/test data, influences model training or preprocessing.

Note: The uploaded course sheet does not provide the actual Titanic CSV data or numerical output values. Therefore, this PDF provides the complete reproducible solution code and methodology rather than inventing dataset-specific statistics.